EE227 – Digital Logic Design

Lecture 6 – 8

Outline

- Boolean Algebra
- Boolean Functions
- Canonical Forms
  - Minterms and Maxterms
- Standard forms
  - Sum of Products (SOP)
  - Product of Sums (POS)
- Digital Logic Gates
- Integrated Circuits
- Quiz 1 Solution

Basic Definitions (1/2)

- Boolean algebra, like any other deductive mathematical system, may be defined with a set of elements, a set of operators, and a number of unproved axioms
- Set
  - A set of elements is any collection of objects, usually having a common property
  - If S is a set, and x and y are certain objects, then the notation $x \in S$ means that x is a member of the set S and $y \notin S$ means that y is not an element of S
  - A set with a denumerable number of elements is specified by braces: A = {1, 2, 3, 4}

Basic Definitions (2/2)

- Binary Operator
  - A binary operator defined on a set S of elements is a rule that assigns, to each pair of elements from S, a unique element from S
  - As an example, consider the relation a *b = c
  - We say that * is a binary operator if it specifies a rule for finding c from the pair (a, b) and also if a, b, c $\in$ S
  - However, * is not a binary operator if a, b $\in$ S, and if c $\notin$ S

The Postulates of a Mathematical System

1) Closure
   - A set is said to be closed if the result obtained after applying a binary operator belongs to the same set
2) Associative Law
   - (x * y) * z = x * (y * z) for all x, y, z, $\in$ S
3) Commutative Law
   - (x * y) = (y * x) for all x, y, $\in$ S
4) Identity Element
   - e * x = x * e = x for every x $\in$ S
5) Inverse
   - x * y = e (y is inverse and e is identity element)
6) Distributive Law
   - x * (y . z) = (x * y) . (x * z)

Axiomatic Definition of Boolean Algebra (1/2)

- In 1854, George Boole developed an algebraic system now called Boolean algebra
- In 1938, Claude E. Shannon introduced a two-valued Boolean algebra called switching algebra that represented the properties of bistable electrical switching circuits
- We will discuss the two valued Boolean algebra only

Axiomatic Definition of Boolean Algebra (2/2)

- Boolean algebra is an algebraic structure defined by a set of elements, B, together with two binary operators, + and . , provided that the following (Huntington) postulates are satisfied
  1. The structure is closed with respect to + and .
  2. The elements 0 and 1 are identity elements w.r.t + and . Respectively
  3. The structure is commutative w.r.t + and .
  4. The operator . is distributive over + and the operator + is distributive over .
  5. For every $x \in B$ there exists $x' \in B$ such that $x + x' = 1$ and $x.x' = 0$
  6. There exist at least two elements $x, y \in B$ such that $x \neq y$

Two Valued Boolean Algebra

- A two-valued Boolean algebra is defined on a set of two elements, B = {0, 1}, with rules for the two binary operators + and . and complement as shown in the following operator tables

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

x	x'
0	1
1	0

- These rules are exactly the same as the AND, OR, and NOT operations, respectively

Practice Problem 1

- Show that the Huntington postulates are valid for the set B = {0, 1} and the two binary operators + and .

- Huntington postulates are:
  1. The structure is closed with respect to + and .
  2. The elements 0 and 1 are identity elements w.r.t + and . Respectively
  3. The structure is commutative w.r.t + and .
  4. The operator . is distributive over + and the operator + is distributive over .
  5. For every $x \in B$ there exists $x' \in B$ such that $x + x' = 1$ and $x.x' = 0$
  6. There exist at least two elements $x, y \in B$ such that $x \neq y$

Basic Theorems and Properties of Boolean Algebra (1/2)

- Duality
  - It states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged
  - If the dual of an algebraic expression is desired, we simply interchange OR and AND operators and replace 1's by 0's and 0's by 1's

Basic Theorems and Properties of Boolean Algebra (2/2)

Postulates and Theorems of Boolean Algebra

Postulate 2	(a)	$x + 0 = x$	(b)	$x \cdot 1 = x$
Postulate 3, commutative	(a)	$x + y = y + x$	(b)	$xy = yx$
Postulate 4, distributive	(a)	$x(y + z) = xy + xz$	(b)	$x + yz = (x + y)(x + z)$
Postulate 5	(a)	$x + x' = 1$	(b)	$x \cdot x' = 0$
Theorem 1	(a)	$x + x = x$	(b)	$x \cdot x = x$
Theorem 2	(a)	$x + 1 = 1$	(b)	$x \cdot 0 = 0$
Theorem 3, involution		$(x')' = x$		
Theorem 4, associative	(a)	$x + (y + z) = (x + y) + z$	(b)	$x(yz) = (xy)z$
Theorem 5, DeMorgan	(a)	$(x + y)' = x'y'$	(b)	$(xy)' = x' + y'$
Theorem 6, absorption	(a)	$x + xy = x$	(b)	$x(x + y) = x$

Practice Problem 2

- Prove the followings
  i. $x + x = x$
  ii. $x \cdot x = x$
  iii. $x + 1 = 1$
  iv. $x \cdot 0 = 0$
  v. $x + xy = x$
  vi. $x(x + y) = x$
  vii. $(x + y)' = x'y'$

Operator Precedence

- The operator precedence for evaluating Boolean expressions
  1. Parentheses
  2. NOT
  3. AND
  4. OR

Boolean Functions (1/2)

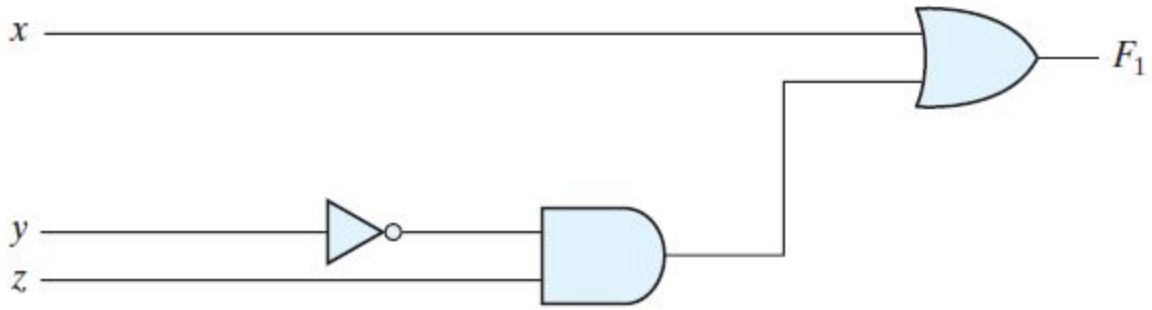
- A Boolean function described by an algebraic expression consists of binary variables, the constants 0 and 1, and the logic operation symbols

$$F_1 = x + y'z$$

- A Boolean function can be represented in a truth table
- The number of rows in the truth table is 2^n, where n is the number of variables in the function
- A Boolean function can be transformed from an algebraic expression into a circuit diagram (also called schematic) composed of logic gates connected in a particular structure

Boolean Functions (2/2)

$F_1 = x + y'z$

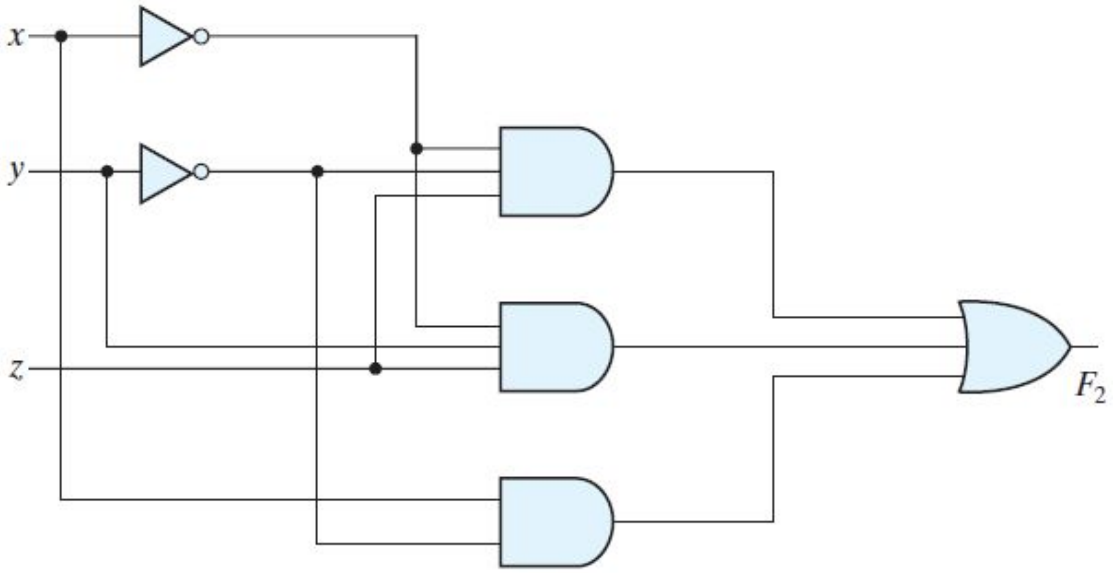


x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Practice Problem 3

- Construct the truth table and circuit diagram for the following Boolean function

$$F_2 = x'y'z + x'yz + xy'$$



x	y	z	x'	y'	x'y'z	x'yz	xy'	F_2
0	0	0	1	1	0	0	0	0
0	0	1	1	1	1	0	0	1
0	1	0	1	0	0	0	0	0
0	1	1	1	0	0	1	0	1
1	0	0	0	1	0	0	1	1
1	0	1	0	1	0	0	1	1
1	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0

Algebraic Manipulation (1/2)

- When a Boolean expression is implemented with logic gates, each term requires a gate and each variable within the term designates an input to the gate
- We define a literal to be a single variable within a term
- The function $F_2 = x'y'z + x'yz + xy'$ has 3 terms and 8 literals
- By reducing the number of terms, the number of literals, or both in a Boolean expression, it is often possible to obtain a simpler circuit
- The manipulation of Boolean algebra consists mostly of reducing an expression for the purpose of obtaining a simpler circuit

Algebraic Manipulation (2/2)

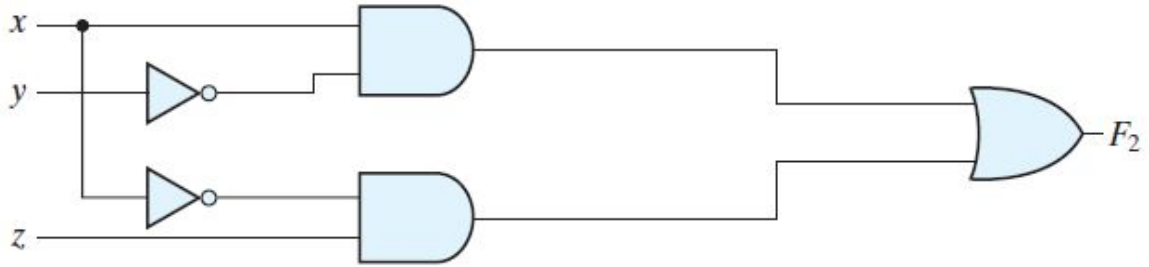
- The simple manipulation consists of applying basic theorems and relations
- The function $F_2 = x'y'z + x'yz + xy'$ can be simplified in the following way

$F_2 = x'y'z + x'yz + xy'$

$F_2 = x'z(y' + y) + xy'$

$F_2 = x'z(1) + xy'$

$F_2 = x'z + xy'$



- The same function is reduced to two terms. The truth table will be the same and the schematic is now simplified

Practice Problem 4

- Simplify the following Boolean functions to a minimum number of literals. Also construct their truth tables and schematics
  i. $x(x' + y)$
  ii. $x + x'y$
  iii. $(x + y)(x + y')$
  iv. $xy + x'z + yz$
  v. $(x + y)(x' + z)(y + z)$

Complement of a Function

- The complement of a function F is F’ and can be obtained by an interchange of 1’s for 0’s and 0’s for 1’s in the value of the function
- The complement of a function may be derived algebraically through DeMorgan’s theorems
- DeMorgan’s theorems for any number of variables resemble the two-variable case

  $(A + B + C + \ldots + F)' = A'B'C'\ldots F'$

  $(ABC \ldots F)' = A' + B' + C' + \ldots + F'$
- The complement of a function is obtained by interchanging AND and OR operators and complementing each literal

Practice Problem 5

- Find the complements of the following functions by applying Demorgan's laws
  i. $F_1 = x'yz' + x'y'z$
  ii. $F_2 = x(y'z' + yz)$

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$
$$F_2' = [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')'(yz)'$$
$$= x' + (y + z)(y' + z')$$
$$= x' + yz' + y'z$$

Another Way to Compute the Complement of a Boolean Function

- A simpler procedure for deriving the complement of a function is to take the dual of the function and complement each literal

Practice Problem 6

- Find the complements of the following functions by using duality
  i. $F_1 = x'yz' + x'y'z$
  ii. $F_2 = x(y'z' + yz)$

1. $F_1 = x'yz' + x'y'z$.
The dual of F_1 is $(x' + y + z')(x' + y' + z)$.
Complement each literal: $(x + y' + z)(x + y + z') = F_1'$

2. $F_2 = x(y'z' + yz)$.
The dual of F_2 is $x + (y' + z')(y + z)$.
Complement each literal: $x' + (y + z)(y' + z') = F_2'$

Canonical Form

- Boolean functions expressed as a sum of minterms or product of maxterms are said to be in canonical form
- Minterms
  - The AND of N variables such that they equals to 1 is called minterm or standard product
  - There are 2^N possible minterms with N variables
  - Minterms are denoted by lower case m
- Maxterms
  - The OR of N variables such that the result is equal to 0 is called maxterm or standard sum
  - There are 2^N possible minterms with N variables
  - Maxterms are denoted by upper case M

Calculating Minterms and Maxterms

- Each minterm is obtained from an AND term of the n variables, with each variable being primed if the corresponding bit of the binary number is a 0 and unprimed if a 1
- Each maxterm is obtained from an OR term of the n variables, with each variable being unprimed if the corresponding bit is a 0 and primed if a 1
- The complement of a minterm is equal to its corresponding maxterm

Minterms and Maxterms for 3 Variables

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Representing a Boolean Function in Canonical Form

- Boolean function can be represented in canonical form in two ways
  - As a sum of minterms
  - As a product of maxterms
- To represent a Boolean function in canonical form all the terms of that function must contain all the variables

Boolean Function in Minterms

- Derive the minterms mathematically
  - Introduce all the variables in each term by applying postulates/theorems of Boolean Algebra
  - Then form the minterms
- Collect the minterms from truth table
  - Generate the truth table directly from the given function
  - Collect the minterms from the truth table

$$F(x, y, z) = \sum(Minterms)$$

Practice Problem 7

- Express the function F = A + B’C as a sum of minterms

Boolean Function in Maxterms

- Derive the maxterms mathematically
  - First convert the function into OR terms
  - Introduce all the variables in each term by applying postulates/theorems of Boolean Algebra
  - Then form the maxterms
- Collect the maxterms from truth table
  - Generate the truth table directly from the given function
  - Collect the maxterms from the truth table

$$F(x, y, z) = \prod(Maxterms)$$

Practice Problem 8

- Express the Boolean function F = xy + x’z as a product of maxterms

Standard Forms

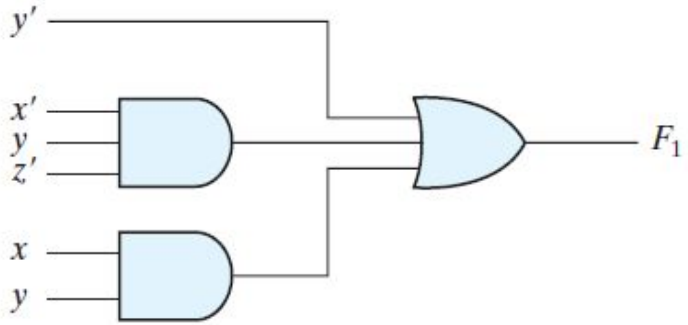
- The canonical forms are seldom used
- The two standard forms to represent Boolean functions are:
  - Sum of Products (SOP)
    - Each term is a sum of 1 or more literals
    - All the terms are combined with AND

      $F_1 = y' + xy + x'yz'$
  - Product of Sums (POS)
    - Each term is a product of 1 or more literals
    - All the terms are combined with OR

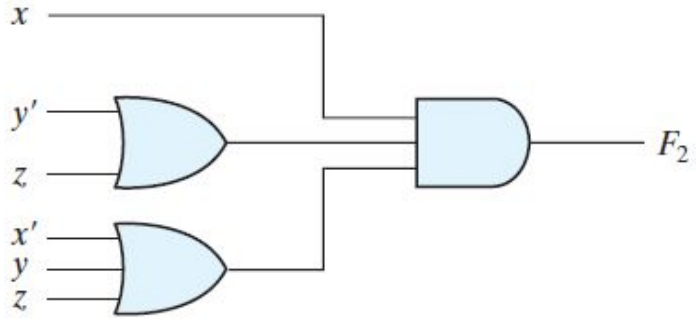
      $F_2 = x(y' + z)(x' + y + z')$
- The standard forms always leads to a two level implementation

2 and 3 Level Implementation

- Two Level Implementation

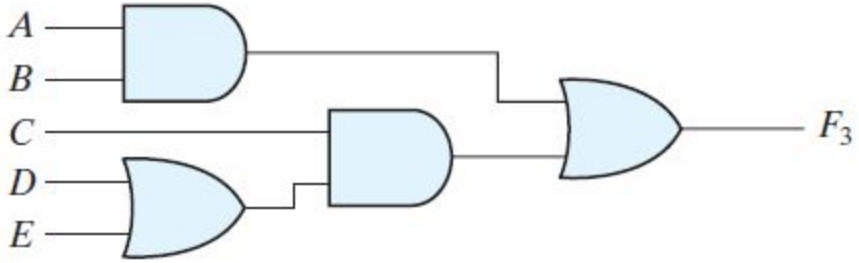


(a) Sum of Products

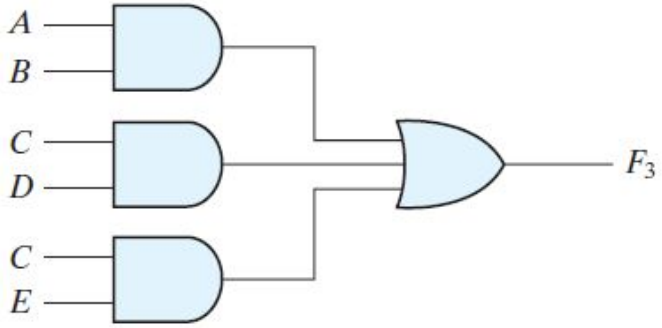


(b) Product of Sums

- Three level implementation

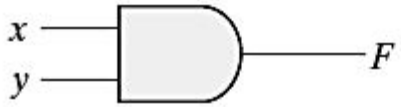
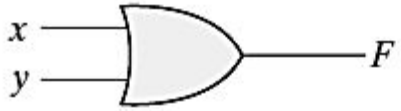
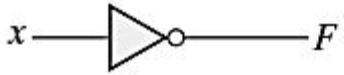
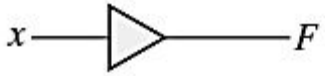


(a) $AB + C(D + E)$

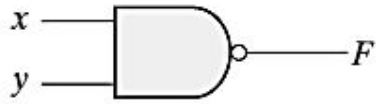
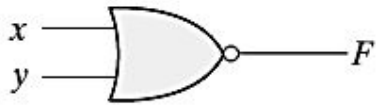
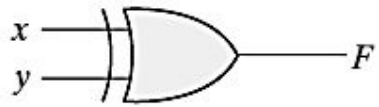
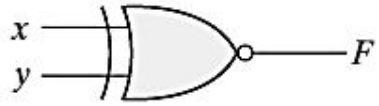


(b) $AB + CD + CE$

Digital Logic Gates (1/2)

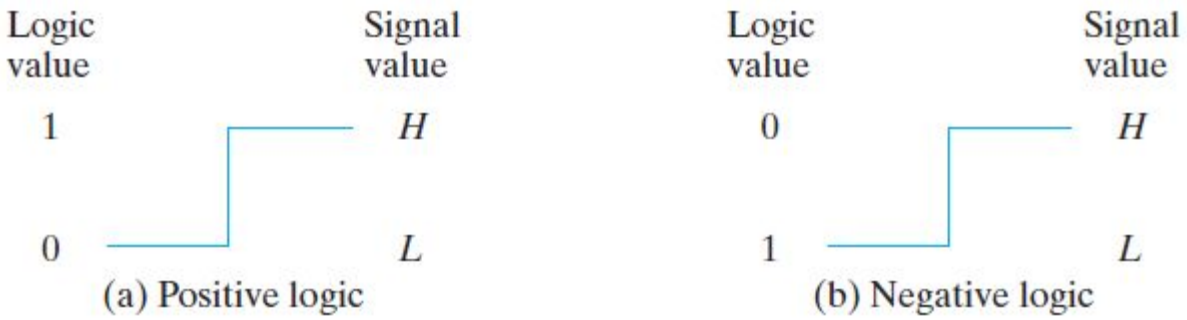
Name	Graphic symbol	Algebraic function	Truth table
AND		$F = x \cdot y$	x y | F 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
OR		$F = x + y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
Inverter		$F = x'$	x | F 0 | 1 1 | 0
Buffer		$F = x$	x | F 0 | 0 1 | 1

Digital Logic Gates (2/2)

Name	Graphic symbol	Algebraic function	Truth table
NAND		$F = (xy)'$	x y | F 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0
NOR		$F = (x + y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1

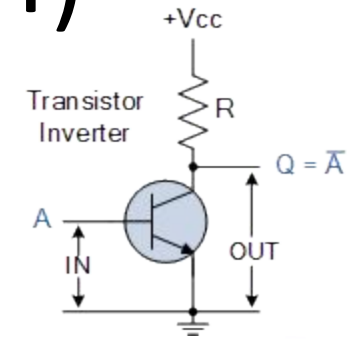
Positive and Negative Logic

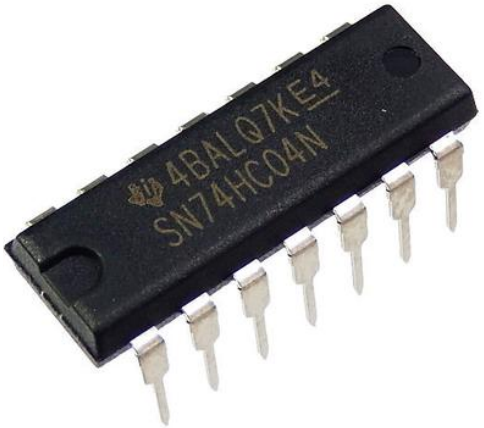
- The binary signal at the inputs and outputs of any gate has one of two values
- One signal value represents logic 1 and the other logic 0
- Since two signal values are assigned to two logic values, there exist two different assignments of signal level to logic value
- Choosing the high-level H to represent logic 1 defines a positive logic system
- Choosing the low-level L to represent logic 1 defines a negative logic system

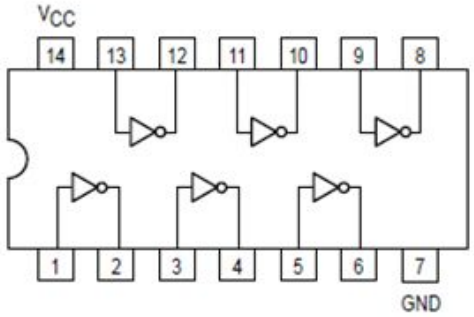


Integrated Circuits (1/4)

- An integrated circuit (IC) is fabricated on a die of a silicon semiconductor crystal, called a chip, containing the electronic components for constructing digital gates
- The various gates are interconnected inside the chip to form the required circuit
- The number of pins may range from 3 on a small IC package to several thousand on a larger package
- Each IC has a numeric designation printed on the surface of the package for identification







Integrated Circuits (2/4)

- Level of Integration
  - Small Scale Integration (SSI)
    - The number of gates is usually fewer than 10
    - AND, OR, XOR etc ICs
  - Medium Scale Integration (MSI)
    - Have a complexity of approximately 10 to 1,000 gates in a single package
    - Decoders, adders, and multiplexers
  - Large Scale Integration (LSI)
    - These devices contain thousands of gates in a single package.
    - They include digital systems such as processors, memory chips, and programmable logic devices
  - Very Large Scale Integration (VLSI)
    - Contain millions of gates within a single package
    - Examples are large memory arrays and complex microcomputer chips

Integrated Circuits (3/4)

- Digital Logic Families
  - The circuit technology of an IC is referred to as a digital logic family
  - TTL (transistor-transistor logic)
    - 50 years old and standard method
  - ECL (emitter coupled logic)
    - For high speed operations
  - MOS (metal oxide semiconductor)
    - Suitable for circuits that need high component density
  - CMOS (complementary metal-oxide semiconductor)
    - CMOS is preferable in systems requiring low power consumption

Integrated Circuits (4/4)

- Fan-out
  - It specifies the number of standard loads that the output of a typical gate can drive without impairing its normal operation
- Fan-in
  - It is the number of inputs available in a gate
- Power dissipation
  - It is the power consumed by the gate that must be available from the power supply
- Propagation delay
  - It is the average transition delay time for a signal to propagate from input to output
- Noise margin
  - It is the maximum external noise voltage added to an input signal that does not cause an undesirable change in the circuit output

Quiz 1

Solution

Quiz 1 (Paper A)

Question Number 1 **[8 Marks]**

Convert $(124.43)_5$ to its binary equivalent

$(124.43)_5 = (1 \times 5^2) + (2 \times 5^1) + (4 \times 5^0) + (4 \times 5^{-1}) + (3 \times 5^{-2}) = (39.92)_{10}$

Now convert 39 and 0.92 to binary separately by using division and multiplication with 2 respectively

$39 = (100111)_2$

$0.92 \approx (1110101100)_2$

So

$(124.43)_5 = (1 \times 5^2) + (2 \times 5^1) + (4 \times 5^0) + (4 \times 5^{-1}) + (3 \times 5^{-2}) = (39.92)_{10} \approx (100111 . 1110101100)_2$

Quiz 1 (Paper A)

Question Number 2 **[1 + 5 + 1 + 3 + 2 = 12 Marks]**

Two decimal numbers are given as A = 387 and B = 189.

a) Write their equivalent BCD codes

b) Add both these numbers by using BCD codes

c) Convert A and B in base 6.

d) Add them in base 6

e) And prove that the answer is same as in decimal

Part (a)

BCD of 387 is 0011 1000 0111

BCD of 189 is 0001 1000 1001

Part (c)

By dividing the number with 6 and collecting the remainders, we get:

$387 = (1443)_6$

$189 = (513)_6$

Part (b)

Carry After Correction							1					1			
Carry		1				1	1		1				1	1	1	1
	3	8	7	0	0	1	1		1	0	0	0		0	1	1	1
+	1	8	9	0	0	0	1		1	0	0	0		1	0	0	1
	5	7	6	0	1	0	1	1	0	0	0	1	1	0	0	0	0
Correction									0	1	1	0		0	1	1	0
				0	1	0	1		0	1	1	1		0	1	1	0
					5					7					6		

Part (d)

Carry	1	1	1	
	1	4	4	3
+		5	1	3
	2	4	0	0

Part (e)

$2400 = (2 \times 6^3) + (4 \times 6^2) + 0 + 0 = (576)_{10}$

Quiz 1 (Paper B)

Question Number 1 **[2 + 8 + 2 = 12 Marks]**

Let A = 24, B = –12 and C = 38

a) Represent them in 2's complement form in binary by using 8 bits

b) Calculate A – B – C by using 2's complement method

c) And verify that your answer is correct.

Part (a)

$A = +24 = (0001\ 1000)_2 \Rightarrow -A = (1110\ 1000)_2$

$B = -12 = (1111\ 0100)_2 \Rightarrow -B = (0000\ 1100)_2$

$C = +38 = (0010\ 0110)_2 \Rightarrow -C = (1101\ 1010)_2$

Part (c)

A – B – C = 1111 1110

2's complement of 1111 1110 is 0000 0010 = -2

So the answer is correct

Part (b)

A – B – C = A + (-B) + (-C)

Carry				1	1				
A		0	0	0	1	1	0	0	0
-B	+	0	0	0	0	1	1	0	0
A + (-B)	=	0	0	1	0	0	1	0	0
-C	+	1	1	0	1	1	0	1	0
A + (-B) + (-C)	=	1	1	1	1	1	1	1	0

Quiz 1 (Paper B)

Question Number 2 **[4 x 2 = 8 Marks]**

The state of a 12-bit register is 100010010111. What is its content if it represents

a) Three decimal digits in BCD

b) Three decimal digits in the excess-3 code

c) Three decimal digits in the 6,3,1,1 code

d) A binary number

(a)	1000 = 8 1001 = 9 0111 = 7 So the contents of register are $(897)_{10}$
(b)	1000 = 5 1001 = 6 0111 = 4 So the contents of register are $(564)_{10}$
(c)	1000 = 6 1001 = 7 0111 = 5 So the contents of register are $(675)_{10}$
(d)	$100010010111 = (2199)_{10}$ So the contents of register are $(2199)_{10}$